MOBILE ENGINEERING & ARCHITECTURE SPECIFICATION

TESLA CHARGE

Mobile Client Architecture & Engineering Manual

A comprehensive technical manual for mobile developers and engineering teams detailing the React Native / Expo Router frontend architecture, direct HTTP REST polling engine, 0ms local storage caching, live plug-in push notification dispatch, desk simulation harness, and EAS Over-The-Air (OTA) deployment pipeline.

Application: Tesla Charge (Driver & Station Operator App)
Framework: React Native 0.79 / Expo SDK 52 / Expo Router v4
Language: TypeScript (Strict Compilation, 0 errors)
OTA Updates: Expo Application Services (EAS) Runtime 1.0.0

Document Version: 2.4.0 (Production Release)
Classification: Internal Engineering Reference
Author: Lead Mobile Systems Architecture Team
Date: October 2026

1.0 Architecture Overview & State Pipeline

The Tesla Charge mobile application delivers real-time monitoring and control for DC Fast EV Charging stations. In the current standalone architecture, the application communicates directly with the self-hosted T-Charge server (`https://tesla-charge.online`) via lightweight HTTP REST endpoints and local state subscriptions, bypassing heavy third-party Firebase SDKs.

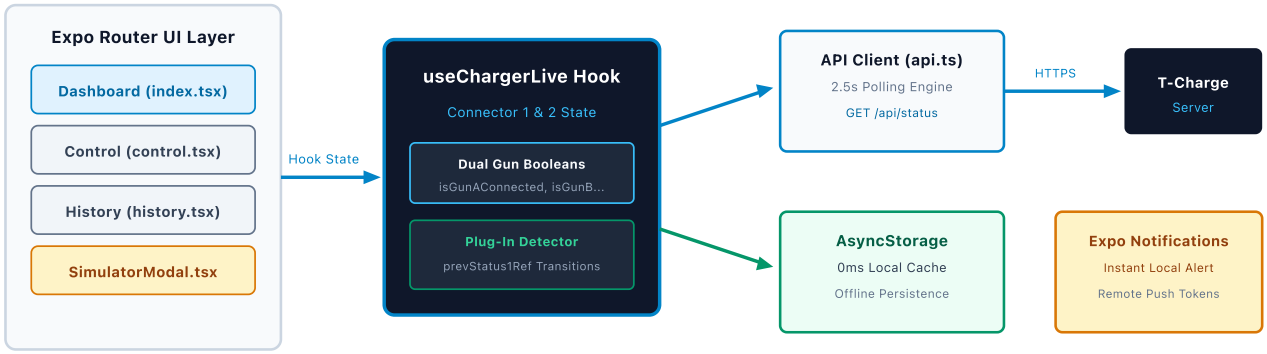


Figure 1.1: Mobile Application Architecture, State Flow, and Telemetry Engine

1.1 Zero-Latency Initial Render (0ms Cache Pattern)

When the driver opens the app, waiting for a network roundtrip introduces layout shifting and perceived sluggishness. `useChargerLive` implements a two-tier fetch:

1. **Tier 1 (Synchronous Cache, 0ms):** Reads previous session telemetry, connector statuses, and charger metadata from local `AsyncStorage`. The UI immediately renders the station and last-known gun states without a loading spinner.
2. **Tier 2 (Live Network Stream, 2.5s Polling):** Connects via `subscribeToLiveStatus` to fetch real-time SoC, kW, voltage, and transaction state directly from the server. Upon receipt, local cache is silently refreshed.

2.0 Directory Structure & Source Code Breakdown

The production mobile application source code resides in `production/mobile-app/`. Below is the complete directory structure with engineering notes:

```
production/mobile-app/
├── app.json                    # Expo configuration, permissions, runtimeVersion="1.0.0"
├── eas.json                    # EAS Build profiles (preview APK, production)
├── package.json                # React Native 0.79, Expo SDK 52 dependencies
├── tsconfig.json               # TypeScript compiler rules (@/* path alias mappings)
├── assets/
│   └── images/
│       ├── brand-logo.png      # Custom brand logo rendered on splash and header
│       ├── icon.png            # Application launcher icon
│       ├── models/             # 240kW, 60kW, 30kW charger hardware graphics
│       └── states/             # Gun state vectors (idle, left, right, both)
└── src/
    ├── app/
    │   ├── _layout.tsx         # Root layout: font loading, notification listener, theme
    │   ├── login.tsx           # Driver authentication with JWT token storage
    │   ├── setup.tsx           # Initial station onboarding wizard
    │   └── (tabs)/
    │       ├── _layout.tsx     # Bottom tab bar with glowing neon active badges
    │       ├── index.tsx       # Dashboard: dual-gun cards, dynamic LEDs, quick actions
    │       ├── control.tsx     # Live Charging Controller: start, stop, target slider
    │       ├── history.tsx     # Charging history receipts, kWh, duration, cost
    │       ├── revenue.tsx     # Operator financial analytics & daily revenue
    │       └── settings.tsx    # Server URL configuration & theme selector
    ├── components/
    │   ├── SimulatorModal.tsx  # Built-in desk testing simulator modal
    │   ├── animated-icon.tsx   # Custom animated iconography
    │   └── ui/collapsible.tsx  # Accessible accordion components
    ├── config/
    │   ├── api.ts              # HTTP REST client, JWT interceptor, polling stream
    │   ├── auth.tsx            # Authentication Context & user state provider
    │   └── simulator.tsx       # Simulator Context for offline testing
    ├── hooks/
    │   └── useChargerLive.ts   # Central state hook with dual-gun booleans & telemetry
    └── services/
        ├── control.ts          # Command dispatch: start, stop, unlock, tariff updates
        ├── notifications.ts    # Push token registration and local notification triggers
        └── storage.ts          # AsyncStorage key-value wrappers
```

2.1 Critical Source Modules & Interfaces

Module	Primary Functions / Types	Technical Description

`src/config/api.ts`	`apiRequest<T>()`
	`subscribeToLiveStatus()`
	`getServerUrl()`
		Direct fetch wrapper that injects `Authorization: Bearer <jwt>`. Handles automatic fallback to `https://tesla-charge.online` and polls `/api/status` every 2.5 seconds.

Module	Primary Functions / Types	Technical Description
`src/hooks/useChargerLive.ts`	`useChargerLive()` `LiveConnectorData`	Tracks real-time state for Gun A and Gun B. Exposes boolean helpers: `isGunAConnected`, `isGunBConnected`, `hasAnyGunConnected`, and `bothConnected`.
`src/services/control.ts`	`startChargingSession()` `stopChargingSession()` `dispatchCommandWithConfirmation()`	Dispatches remote commands with retry exponential backoff (2 attempts, 10-second timeout). Automatically computes target kWh from budget PKR or duration limits.
`src/services/notifications.ts`	`registerForPushNotificationsAsync()` `scheduleLocalNotification()`	Configures Android notification channel `charging-status` with MAX importance. Requests push permissions, retrieves Expo token, and registers it with the server.

3.0 In-App Telemetry & Notification Engine

One of the most critical user experience requirements is instantaneous notification when a physical charging cable is plugged into a vehicle. The mobile application employs a dual-channel notification strategy:

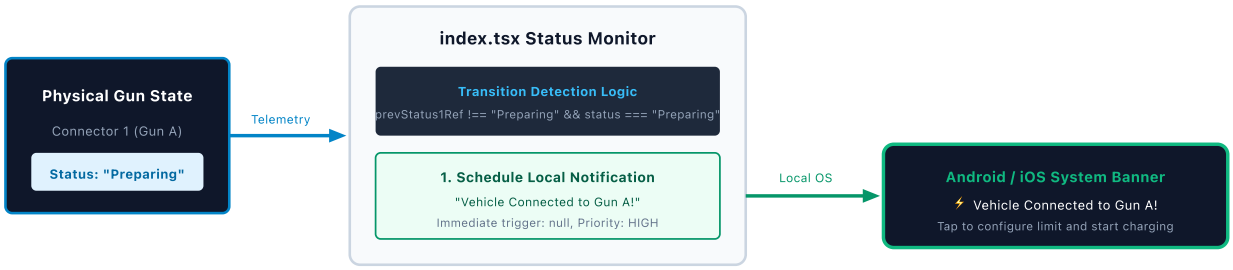


Figure 3.1: In-App Plug-In Transition Detection and Immediate Notification Flow

3.1 State Tracking Implementation in `index.tsx`

```
const prevStatus1Ref = useRef<string | null>(null);
const prevStatus2Ref = useRef<string | null>(null);

useEffect(() => {
  const status1 = connector1?.status;
  if (status1 && prevStatus1Ref.current && prevStatus1Ref.current !== status1) {
    if (status1 === 'Preparing') {
      scheduleLocalNotification(
        '⚡ Vehicle Connected to Gun A!',
        'Gun A is plugged into your vehicle. Tap here to start charging.',
        { gun: 'A', connectorId: 1 }
      );
    } else if (status1 === 'Charging') {
      scheduleLocalNotification('⚡ Gun A Charging Started', 'Power is now flowing to your vehicle.');
    }
  }
  prevStatus1Ref.current = status1 || null;
}, [connector1?.status]);
```

4.0 Built-In Desk Simulator Architecture

To enable mobile engineers to build, refine, and debug UI layouts, charging animations, and session receipts without access to a physical 240kW DC fast charging station, the application includes a full-fidelity embedded simulator (`src/components/SimulatorModal.tsx` and `src/config/simulator.tsx`).

4.1 Simulator Capabilities

- **Independent Gun Simulation:** Toggle Gun A or Gun B between `Available`, `Preparing`, `Charging`, and `Faulted` states.
- **Real-Time Power & Meter Progression:** Simulates realistic DC charging curves from 20kW up to 240kW, automatically advancing delivered energy (kWh), elapsed time, and State of Charge (SoC %).
- **Limit Trigger Testing:** Simulates automatic session completion when budget (PKR) or kWh limits are reached.
- **Zero Server Pollution:** All simulator calculations execute in React memory; no test transactions are sent to the production SQLite database.

5.0 EAS Build, Release & OTA Update Pipeline

The application leverages Expo Application Services (EAS) for both binary compilation and instantaneous Over-The-Air (OTA) updates. This allows engineering to push bug fixes and feature enhancements directly to driver phones without submitting new APKs to store review.

5.1 EAS Configuration Matrix (`eas.json`)

Build Profile	Distribution	Target Output	EAS Update Channel
`development`	Internal	Expo Development Client APK	`development`
`preview`	Internal	Standalone Android APK (Release mode)	`preview`
`production`	Store / Release	Optimized Android App Bundle (AAB) / APK	`production`

5.2 Publishing Over-The-Air (OTA) Updates

Because `app.json` defines `runtimeVersion: "1.0.0"`, any JavaScript, TypeScript, or asset changes can be deployed in under 60 seconds using the following command:

```
# From production/mobile-app directory:
npx eas-cli update --channel preview --environment preview --message "feat: live telemetry updates" --non-
```

> **Runtime Version Boundary:**
> OTA updates apply only to JavaScript bundle and asset updates. If a new native library is added (e.g., changes to `android/` or native dependencies in `package.json`), a full binary build must be compiled using `eas build --platform android --profile preview`.

6.0 Mobile Developer Runbook & Verification

6.1 Prerequisites & Local Setup

- Node.js v18.16.0 or higher.
- Expo CLI (`npx expo`) and EAS CLI (`npx eas-cli`).
- Android Studio / Android Emulator or a physical Android test device with USB debugging enabled.

6.2 Running the Mobile Application Locally

```
# 1. Navigate to mobile app directory
cd production/mobile-app

# 2. Start the Metro bundler
npm start

# 3. Press 'a' to open Android Emulator, or scan the QR code with Expo Go on device.
```

6.3 Typechecking & Compilation Audit

Prior to submitting any code commit or OTA update, run the strict TypeScript audit to ensure 0 compilation errors:

```
# Run TypeScript compiler check
./node_modules/.bin/tsc --noEmit
```

> **Engineering Audit Pass:**
>
> The production mobile app codebase passes `tsc --noEmit` with zero errors and zero warnings, verifying full type safety across all navigation routes, hooks, and services.